

MATH40082 (Computational Finance)
Main Assignment: Finite Difference Methods

10458528

March 2024

Abstract

In this project the value of a coupon paying bond as well as an American put option were valued using the Crank-Nicolson finite difference method. The value of the bond at the interest rate r_0 was calculated to be $B(r_0, 0; T) = 49.90797$. The value of the American option was calculated to be $V(r_0, 0; T_1, T) = 0.741797$.

1 Theory

1.1 Coupon Bonds with stochastic Interest Rates

This report focuses on the valuation of coupon bonds, which provide continuous payouts in the form of coupons and yield a specific amount, known as the face value, at the maturity date. We denote the value of the bond at time t , maturing at time T , as $B(r, t; T)$, where r represents the stochastic interest rate. The risk-neutral process followed by the interest rate is given by

$$dr = \kappa(\theta e^{\mu t} - r)dt + \sigma r^\beta dW, \quad (1)$$

where the constants are given by $\kappa = 0.08116$, $\theta = 0.0409$, $\mu = -0.0222$, $\sigma = 0.251$, $\beta = 0.527$. The market value of the coupon bond $B(r, t; T)$ satisfies the following PDE

$$\frac{\partial B}{\partial t} + \frac{1}{2}\sigma^2 r^{2\beta} \frac{\partial^2 B}{\partial r^2} + \kappa(\theta e^{\mu t} - r) \frac{\partial B}{\partial r} - rB + Ce^{-\alpha t} = 0, \quad (2)$$

if the bond pays out continuous coupons at the rate

$$Ce^{-\alpha t} \quad (3)$$

for the constants C and α that are specified by the bond contract in this report to be $C = 1.07$ and $\alpha = 0.01$. The domain of the problem is $r \in [0, \infty)$ and $t < T$. The Boundary conditions on $B(r, t; T)$ are given by

$$B(r, t = T; T) = F; \quad (4)$$

$$\frac{\partial B}{\partial t} + \kappa\theta e^{\mu t} \frac{\partial B}{\partial r} + Ce^{-\alpha t} = 0 \text{ at } r = 0; \quad (5)$$

$$B(r, t; T) \rightarrow 0 \text{ as } r \rightarrow \infty. \quad (6)$$

The last condition is known as the Dirichlet condition, and it can be replaced by the weaker Neumann condition

$$\frac{\partial B}{\partial r} \rightarrow 0 \text{ as } r \rightarrow \infty \quad (7)$$

1.2 Options on a Bond

If we consider a put option V , then we can denote the value of the option to sell the coupon bond $B(r, t; T)$ at time T_1 by $V(r, t; T_1, T)$. It can be shown that, on the domain $r \in [0, \infty)$ and $t < T_1$, that the option value $V(r, t; T_1, T)$ follows the PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 r^{2\beta} \frac{\partial^2 V}{\partial r^2} + \kappa(\theta e^{\mu t} - r) \frac{\partial V}{\partial r} - rV = 0. \quad (8)$$

For an American put option that expires at $t = T_1$ we have

$$V(r, t = T_1; T_1, T) = \max(X - B(r, T_1; T), 0) \quad (9)$$

and if the option to sell the bond is exercised early

$$V(r, t; T_1, T) = X - B(r, t; T) \quad (10)$$

According to the no-arbitrage principle and early exercise, the option must satisfy the boundary conditions

$$V(r, t; T_1, T) \geq \max(X - B(r, t; T), 0), \text{ for } t \leq T_1; \quad (11)$$

$$\frac{\partial V}{\partial t} + \kappa \theta e^{\mu t} \frac{\partial V}{\partial r} = 0 \text{ at } r = 0; \quad (12)$$

$$V(r, t; T_1, T) \rightarrow X - B(r, t; T) \text{ as } r \rightarrow \infty. \quad (13)$$

If we instead have a European put option then

$$V(r, t = T_1; T_1, T) = \max(X - B(r, t; T), 0) \quad (14)$$

$$\frac{\partial V}{\partial t} + \kappa \theta e^{\mu t} \frac{\partial V}{\partial r} = 0 \text{ at } r = 0; \quad (15)$$

$$V(r, t; T_1, T) \rightarrow 0 \text{ as } r \rightarrow \infty \quad (16)$$

or

$$\frac{\partial V}{\partial r} \rightarrow 0 \text{ as } r \rightarrow \infty \quad (17)$$

2 The Crank Nicolson Methods

We can use finite difference methods in the form of the Crank-Nicolson scheme to approximate the value of derivatives within our problem. The Crank-Nicolson scheme works by evaluating the derivatives at $V(S, t + \Delta t/2)$, resulting in a theoretical error in the time derivative of $(\Delta t)^2$. If we have some function $B(r, t)$ over a domain $r \in [0, r_{max}]$ and $t \in [0, T]$, then we can discretise the domains by defining j_{max} subintervals of length $\Delta r = r_{max}/j_{max}$ and i_{max} subintervals of length $\Delta t = T/i_{max}$. Therefore the nodes of the grid are given by $0, \Delta r, 2\Delta r, \dots, j_{max}\Delta r$ and $0, \Delta t, \dots, i_{max}\Delta t$. The value of $B(r, t)$ at each node in the grid is then given by $B(j\Delta r, i\Delta t) = B_j^i$. The approximations given by the Crank-Nicolson scheme in terms of B_j^i are

$$\frac{\partial B}{\partial t} \approx \frac{B_j^{i+1} - B_j^i}{\Delta t}; \quad (18)$$

$$\frac{\partial B}{\partial r} \approx \frac{1}{4\Delta r} (B_{j+1}^i - B_{j-1}^i + B_{j+1}^{i+1} - B_{j-1}^{i+1}); \quad (19)$$

$$\frac{\partial^2 B}{\partial r^2} \approx \frac{1}{2\Delta r^2} (B_{j+1}^i - 2B_j^i + B_{j-1}^i + B_{j+1}^{i+1} - 2B_j^{i+1} + B_{j-1}^{i+1}), \quad (20)$$

$$B \approx \frac{1}{2} (B_j^i + B_j^{i+1}); \quad (21)$$

These approximations can be substituted into the PDE of equation (2) and rearranged such that the values corresponding to B^i are on one side and the values corresponding to B^{i+1} are on the other. This is done since the former values will be the unknowns whilst the latter values will be known values. For the values of r and t we substitute $j\Delta r$ and $(i + 1/2)\Delta t$ and once rearranged we get the formula

$$\begin{aligned}
& \frac{1}{4}B_{j-1}^i \left(\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} - \kappa j \right) + B_j^i \left(\frac{1}{\Delta t} + \frac{1}{2} \sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{j\Delta r}{2} \right) \\
& + \frac{1}{4}B_{j+1}^i \left(-\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} - \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} + \kappa j \right) \\
& = \frac{1}{4}B_{j-1}^{i+1} \left(-\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} - \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} + \kappa j \right) \\
& \quad - B_j^{i+1} \left(\frac{1}{\Delta t} + \frac{1}{2} \sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{j\Delta r}{2} \right) \\
& \quad + \frac{1}{4}B_{j+1}^{i+1} \left(\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} - \kappa j \right) + C e^{-\alpha(i+1/2)\Delta t}.
\end{aligned} \tag{22}$$

The problem can be rewritten in terms of a matrix equation as

$$\begin{bmatrix} b_0 & c_0 & 0 & 0 & \cdot & \cdot & \cdot & \cdot & 0 \\ a_1 & b_1 & c_1 & 0 & \cdot & \cdot & \cdot & \cdot & \cdot \\ 0 & a_2 & b_2 & c_2 & 0 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & a_3 & b_3 & c_3 & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & a_j & b_j & c_j & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 0 & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & a_{jmax} & b_{jmax} \end{bmatrix} \begin{bmatrix} B_0^i \\ B_1^i \\ B_2^i \\ B_3^i \\ \vdots \\ B_{jmax-1}^i \\ B_{jmax}^i \end{bmatrix} = \begin{bmatrix} d_0^i \\ d_1^i \\ d_2^i \\ d_3^i \\ \vdots \\ d_{jmax-1}^i \\ d_{jmax}^i \end{bmatrix}, \tag{23}$$

where for $1 \leq j < jmax$, the entries are given by

$$a_j = \frac{1}{4} \left(\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} - \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} + \kappa j \right) \tag{24}$$

$$b_j = - \left(\frac{1}{\Delta t} + \frac{1}{2} \sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{j\Delta r}{2} \right) \tag{25}$$

$$c_j = \frac{1}{4} \left(\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} - \kappa j \right) \tag{26}$$

$$d_j = \frac{1}{4} B_{j-1}^{i+1} \left(-\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} - \kappa j \right) \tag{27}$$

$$+ B_j^{i+1} \left(-\frac{1}{\Delta t} + \frac{1}{2} \sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{j\Delta r}{2} \right) \tag{28}$$

$$+ \frac{1}{4} B_{j+1}^{i+1} \left(-\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} - \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} + \kappa j \right) \tag{29}$$

$$- C e^{-\alpha(i+1/2)\Delta t}. \tag{30}$$

The boundary condition at $r = 0$ given by equation (5) can be implemented at the values given by $j = 0$ by substituting approximations for the derivatives into the boundary condition, however a one sided finite difference approximation must be used for the derivative with respect to r since there is no B_{-1}^i term. This gives

$$b_0^i = \frac{1}{\Delta t} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} \tag{31}$$

$$c_0^i = -\frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} \tag{32}$$

$$d_0^i = \frac{1}{\Delta t} B_0^{i+1} + C e^{-\alpha(i+1/2)\Delta t}. \tag{33}$$

The Dirichlet boundary condition is implemented as

$$a_{jmax}^i = 0, \quad (34)$$

$$b_{jmax}^i = 1, \quad (35)$$

$$d_{jmax}^i = 0. \quad (36)$$

whilst the Neumann Boundary condition is implemented as

$$a_{jmax}^i = -\frac{1}{\Delta r}, \quad (37)$$

$$b_{jmax}^i = \frac{1}{\Delta r}, \quad (38)$$

$$d_{jmax}^i = 0. \quad (39)$$

The Crank-Nicolson scheme can also be implemented to discretise and solve equation (8) for the European put option value V . Applying the same approximations for the derivatives results in another matrix equation in the form of equation (22) with entries for $1 \leq j < jmax$ given by

$$a_j = \frac{1}{4} \left(\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} - \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} + \kappa j \right) \quad (40)$$

$$b_j = - \left(\frac{1}{\Delta t} + \frac{1}{2} \sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{j \Delta r}{2} \right) \quad (41)$$

$$c_j = \frac{1}{4} \left(\sigma^2 j^{2\beta} r^{2(\beta-1)} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} - \kappa j \right) \quad (42)$$

$$d_j = \frac{1}{4} V_{j-1}^{i+1} \left(-\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} - \kappa j \right) \quad (43)$$

$$+ V_j^{i+1} \left(-\frac{1}{\Delta t} + \frac{1}{2} \sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} + \frac{j \Delta r}{2} \right) \quad (44)$$

$$+ \frac{1}{4} V_{j+1}^{i+1} \left(-\sigma^2 j^{2\beta} \Delta r^{2(\beta-1)} - \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} + \kappa j \right) \quad (45)$$

$$(46)$$

The boundary condition at $r = 0$ is given by

$$b_0^i = \frac{1}{\Delta t} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} \quad (47)$$

$$c_0^i = -\frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} \quad (48)$$

$$d_0^i = \frac{1}{\Delta t} V_0^{i+1}. \quad (49)$$

and the boundary condition of equation (16) at $r = r_{max}$ is given by

$$a_{jmax}^i = 0, \quad (50)$$

$$b_{jmax}^i = 1, \quad (51)$$

$$d_{jmax}^i = 0 \quad (52)$$

The boundary condition for the American option at $r=0$ is given by

$$b_0^i = \frac{1}{\Delta t} + \frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} \quad (53)$$

$$c_0^i = -\frac{1}{\Delta r} \kappa \theta e^{\mu(i+1/2)\Delta t} \quad (54)$$

$$d_0^i = \frac{1}{\Delta t} V_0^{i+1}. \quad (55)$$

whilst at $r = r_{max}$

$$a_{j_{max}}^i = 0, \quad (56)$$

$$b_{j_{max}}^i = 1, \quad (57)$$

$$d_{j_{max}}^i = X - B(j_{max}\Delta r, i\Delta t; T) \quad (58)$$

3 Results

3.1 Coupon Bonds

The first result calculated was that of the value of the coupon bond $B(r, 0, T)$ with $r_{max} = 1$ and $i_{max} = j_{max} = 100$ by using the Crank-Nicolson scheme with the Dirichlet Boundary Condition. The values $B_j^{i_{max}}$ were initially set to be equal to F in line with equation (4). Subsequently, solving the matrix equation (22) then gives us values for $B_j^{i_{max}-1}$. This process can be repeated in an iterative process until we have the values of B_j^0 . Linear interpolation was used to estimate values of $B(r, 0, T)$ in between the grid points, such that the value of $B(r_0, 0, T) = 49.907972$ could be calculated for $r_0 = 0.0275$.

The value of r_{max} had to be chosen to be suitably large to accurately represent the value of B at infinity, whilst remaining small enough to maintain stability and minimise computation time. The values of i_{max} and j_{max} determine the grid spacing, and therefore since j_{max} is related to r_{max} it must be kept at a constant ratio to not affect Δr and convergence. The error in the Crank-Nicolson scheme is proportional to Δt^2 and Δr^2 , so the higher the values of i_{max} and j_{max} , the smaller the error becomes.

The values of $B(r, 0; T)$ were then calculated in the same fashion, however, this time using the Neumann boundary condition. The same values of $r_{max} = 1$ with $i_{max} = j_{max} = 1000$ were used. The values of $B(r, 0; T)$ across a range of values of r are plotted in Figure 1 for the use of both boundary conditions. The figure shows that for small values of r the two boundary conditions result in equal values of $B(r, 0; T)$, however as r approaches r_{max} the two solutions do not match. The solution from the Dirichlet boundary condition is forced towards $B(r, 0; T) = 0$ rapidly as r nears r_{max} and so the solution diverges from the behaviour we expect to see, that is, forming an asymptote and diverging to $B(r, 0; T) = 0$ at infinity. Conversely, the solution from the Neumann boundary condition produces the behaviour that we expect, and it can be seen that it appears as though the value of $B(r, 0; T)$ is approaching zero at a slow enough pace to resemble it approaching zero as $r \rightarrow \infty$.

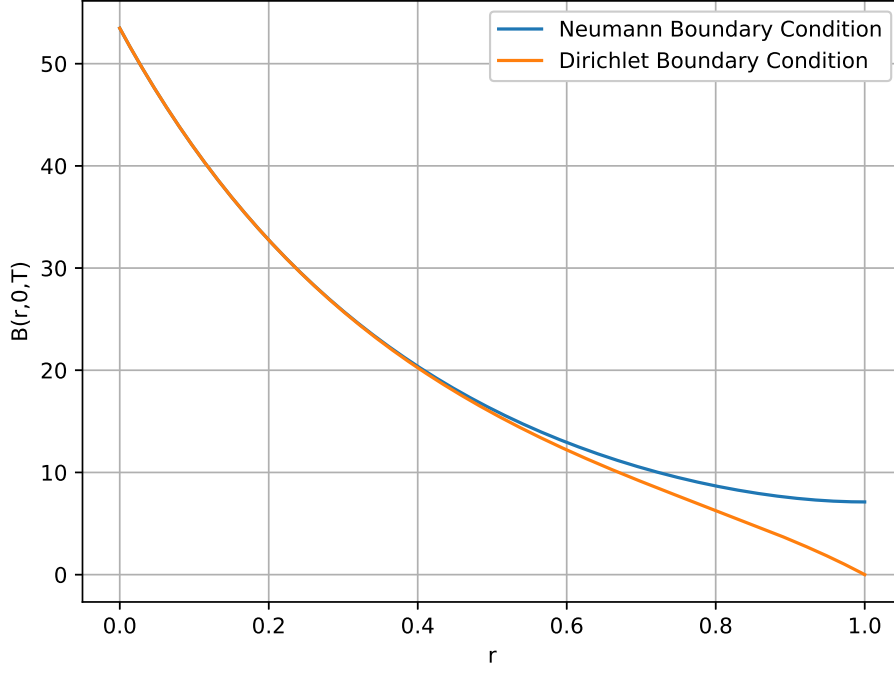


Figure 1: A plot of the value of the functions P and V against the interest rate r .

Two experiments were performed to investigate how the values of i_{max} and j_{max} affected the accuracy of the solution $B(r, 0; T)$. Firstly, the values for $B(r, 0; T)$ were calculated for a range of values of i_{max} from $i_{max} = 200$ to $i_{max} = 204800$, doubling each time, whilst keeping $j_{max} = 100$. Secondly, the values for $B(r, 0; T)$ were calculated for a range of values of j_{max} from $j_{max} = 200$ to $j_{max} = 204800$ whilst keeping $i_{max} = 100$. In both cases, $r_{max} = 1$ was kept constant.

i_{max}	$B(r_0, 0, T)$	Time (s)	Difference	Difference Ratio
200	49.9115172916	0.16925358772277	-	-
400	49.9117576568	0.33841824531555	0.00024036514994207	-
800	49.9118781095	0.6656494140	0.00012045275028071	1.995514003473554
1600	49.91193840340	1.3391003608703	6.029385456685e-05	1.997761648281545
3200	49.91196856717	2.6713509559631	3.0163770908586e-05	1.9988831883644558
6400	49.9119836532	5.3330442905426	1.5086093782201e-05	1.999442091774169
12800	49.9119911974	10.82979941368	7.544133922010e-06	1.9997118208873366
25600	49.9119949697	21.39764094352	3.7723065204886e-06	1.9998729904465289
51200	49.91199685594	43.05422592163	1.886232936954e-06	1.999915517635056
102400	49.9119977995	85.98855233192	9.436402024221024e-07	1.9988899711060273
204800	49.9119982710	172.66677045822	4.714859613841327e-07	2.001417390354265

Table 1: Values for i_{max}

j_{max}	$B(r_0, 0, T)$	Time (s)	Difference	Difference Ratio
200	49.9086575083	0.18475627899169	-	-
400	49.90766075144	0.3512148857116	-0.0009967568945583594	-
800	49.9077793899	0.6897263526916	0.00011863850897952943	-8.401630323340843
1600	49.9078632214	1.3782658576965	8.383145891599497e-05	1.415202723579147
3200	49.9079111368	2.7613885402679	4.7915478269544565e-05	1.7495694907688915
6400	49.9079328347	5.542813301086	2.1697892748306913e-05	2.2083010007173813
12800	49.90793991585	11.216286897659	7.081068467584828e-06	3.064211686080125
25600	49.9079411596	22.32872700691	1.2438396339575775e-06	5.6929111070811365
51200	49.9079410350	44.96781539916	-1.2462195542184418e-07	-9.980902881415973
102400	49.90794089942	90.19935321807	-1.3564574885549519e-07	0.9187310068567297
204800	49.9079408675	181.60391807556	-3.186941199828652e-08	4.256299076455765

Table 2: Values for j_{max}

For the Crank-Nicolson scheme, if the method is order $\mathcal{O}(dt)$ with

$$V_{approx} = V * + \frac{A}{i_{max}} + \dots \quad (59)$$

then the ratio of successive differences should converge towards 2 as i_{max} is doubled. This can be seen to be the case within the rightmost column of Table 1, and so the Crank-Nicolson scheme is confirmed to be of order $\mathcal{O}(dt)$ in its implementation for this project. The theoretical error in the time derivative is expected to be $\mathcal{O}(dt^2)$, however, this is reduced by the application of first-order difference approximations for the derivative at the boundary. If the method is order $\mathcal{O}(dr^2)$ with

$$V_{approx} = V * + \frac{B}{j_{max}^2} \quad (60)$$

then the ratio of successive differences should converge towards 4 as j_{max} doubles. Although less obvious, this appears to possibly be the case within the rightmost column of Table 2 and therefore the implementation of the Crank-Nicolson scheme is of order $\mathcal{O}(dr^2)$

r_{max}	j_{max}	Time (s)	$B(r_0, 0, T)$
1	100	0.09050464630126953	49.911038726640726
2	200	0.17660999298095703	49.911038677966715
4	400	0.35125088691711426	49.911038677966566
8	800	0.7093000411987305	49.911038677966566
16	1600	1.4096381664276123	49.911038677966566
32	3200	2.843999147415161	49.911038677966566
64	6400	5.774275064468384	49.911038677966566
128	12800	11.280154466629028	49.911038677966566
256	25600	22.84798836708069	49.911038677966566
512	51200	44.95229506492615	49.911038677966566

Table 3: Values for different r_{max} and j_{max}

Increasing both i_{max} and j_{max} shows convergence towards a solution of $B(r_0, 0, T) = 49.91$ whilst, however, requiring longer times to complete the calculation. Increasing r_{max} above $r_{max} = 4$ has no observable effect on the accuracy of the result whilst considerably increasing computation time. When investigating the effect of r_{max} on the solution the value of j_{max} was kept proportional to maintain the length dr .

3.2 Options on a Bond

The second half of this project was concerned with calculating the value of an American put option V with $T_1 = 0.9839$ and $X = 49.6$. The solution is slightly more complex to obtain since the Crank-Nicolson scheme had to first be used to calculate values of B_j at $t = T_1$, which can then be used in the boundary condition for the European option V . The Crank-Nicolson scheme can then be used again on V to calculate $V(r, 0; T_1, T)$. The option of early exercise also has to be taken into account by incorporating equation (11) into the simulation. Since equation (11) relies on the values of $B(r, t; T)$, we have to calculate its values simultaneously with $V(r, t; T_1, T)$. At each step of the backward algorithm the value of $X - B_j^i$ is calculated and if the values V_j^i are smaller than this value then we set $V_j^i = X - B_j^i$.

Alternatively, we incorporate equation (11) via the penalty method, in which we alter the finite difference method at each time step for the values V_j^i that satisfy $V_j^i < X - B_j^i$ by

$$\hat{b}_j^i = b_j^i - \rho \quad (61)$$

$$\hat{d}_j^i = d_j^i - \rho(B_j^i - X) \quad (62)$$

for large ρ . The linear system is then solved and the previous step is repeated until the difference between consecutive solutions is reduced below the tolerance, set to $1/\rho$.

Since the put option is exercised at $T_1 = 0.9838$, there is little chance that this point naturally lies upon a grid point. For this reason, once the grid has been set up, the entire grid is shifted by a constant value such that the nearest grid point to T_1 coincides with T_1 . Consequently, the size of the first and final time steps will become smaller or larger depending on if the internal grid points are shifted upwards or downwards.

The value of the American put option $V(r, 0; T_1, T)$ was calculated for a range of values of r and a plot was created of $V(r, 0; T_1, T)$ along with $V(r, T_1; T_1, T)$ against r as can be seen in Figure 2. The values of i_{max} and j_{max} were set to $i_{max} = j_{max} = 1000$ and $r_{max} = 1.0$. At every point we have that $V(r, T_1; T_1, T) > V(r, 0; T_1, T)$.

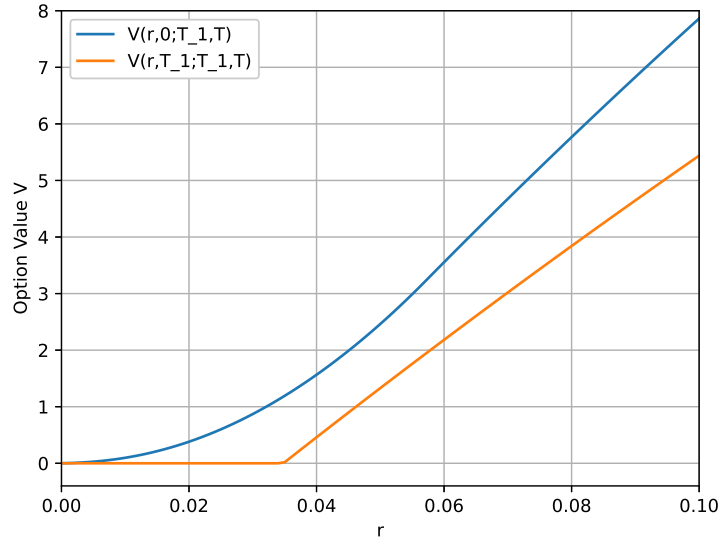


Figure 2: A plot of the value of the functions P and V against the interest rate r .

To find the minimum value of r at time $t = T_1$, a linear fit was calculated for the part of $V(r, T_1; T_1, T)$ that isn't equal to zero. The minimum value of r is then the value at which this linear fit crosses the r axis. The value of $V(r, T_1; T_1, T)$ is only approximately linear for $0.04 < r < 0.1$ and so this section of $V(r, T_1; T_1, T)$ was isolated for the linear fit. The linear fit was calculated to be $V(r, T_1; T_1, T) \approx 82.9573954r - 2.81029114$ which gives an r axis intercept of $r = 0.0338763$, this being the minimum value for r at $t = T_1$.

The value of $V(r_0, 0; T_1, T)$ was calculated using both the standard method and the penalty method with $i_{max} = j_{max} = 100$ and $r_{max} = 1.0$. The standard method produced the value of $V(r_0, 0; T_1, T) = 0.737739$ whilst inclusion of the penalty method gave a value of $V(r_0, 0; T_1, T) = 0.741797$.

Using the penalty method and a value of $r_{max} = 1.0$, the effect of changing i_{max} and j_{max} was investigated. The value of $j_{max} = 100$ was held constant whilst i_{max} was increased by multiples of two and then the opposite was carried out to test j_{max} . The ratio of the difference in each successive solution to $V(r_0, 0; T_1, T)$ was calculated and with this ratio converging to two when doubling i_{max} we see that the numerical scheme used is $\mathcal{O}(dt)$. When increasing the value of j_{max} , the ratio of the differences does not appear to converge to any specific value.

i_{max}	$V(r_0, 0; T_1, T)$	Time (s)	Difference Ratio
200	0.7417542900436302	0.26671767234802246	1.3481553304412701
400	0.7416109021701373	0.5324106216430664	-5173.061514716359
800	0.7416016777643493	1.0597572326660156	15.544402185743389
1600	0.7416167385845941	2.0936710834503174	-0.6124769858505275
3200	0.7416026344832372	4.088428974151611	-1.0678326724743412
6400	0.7416051688173528	8.162762641906738	-5.5652099183315995
12800	0.7416044688373619	16.33886432647705	-3.620580799981896
25600	0.7416041203132289	32.40330195426941	2.008411827751147
51200	0.7416039487188641	64.76700568199158	2.0310931164446346
102400	0.7416038676669345	129.83957314491272	2.117091666268521
204800	0.7416038366896534	259.8823821544647	2.616495912902477

Table 4: Values for different imax

j_{max}	$V(r_0, 0; T_1, T)$	Time (s)	Difference Ratio
200	0.7326251448603814	0.2605416774749756	1.3649545159831675
400	0.7281025606036527	0.5107426643371582	-161.99259168480577
800	0.728569831358027	1.022761344909668	-9.678723126563222
1600	0.7286693390795254	2.018423080444336	4.695824076144199
3200	0.5531060352237482	4.01740574836731	-0.0005667911192882539
6400	0.5531054448844482	7.94636607170105	297393.8950971411
12800	0.553105382120155	15.925260782241821	9.405655187590757
25600	0.553105311884026	31.84097123146057	0.8936183425742706
51200	0.5531053396682931	63.61617398262024	-2.5279100847871536
102400	0.5531053322694803	127.15504956245422	-3.7552331594119983
204800	0.5531053389816909	255.21043586730957	-1.1022915197401286

Table 5: Values for different jmax

Appendix A Source code listings

A.1 finite_difference.py

```

1 import numpy as np
2 import scipy
3 from scipy.linalg import solve_banded
4 import matplotlib.pyplot as plt
5 import time
6
7 T = 3
8 T_1 = 0.9839
9 X = 49.6
10 F = 51
11 r_0 = 0.0275
12 imax = 1000
13 jmax = 1000
14 r_max = 3.0
15
16 Option_Type = "put" # "call" or "put"
17
18 #sigma, kappa, theta, mu, alpha, beta, C
19 constants = (0.251, 0.08116, 0.0409, -0.0222, 0.01, 0.653, 1.07)
20
21
22 def Bond_matrix(B0ld, i, dr, dt, constants, imax, jmax, rmax, boundary = '
    Dirichlet'):
23
24     sigma, kappa, theta, mu, alpha, beta, C = constants
25
26     A_banded = np.zeros(shape=(3, jmax+1))
27     u, l= 1, 1
28     l_and_u = (l, u)
29
30     # b_0 V_0 + c_0 V_1 = d_0
31     A_banded[1][0] = 1/(dt) + (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) # b_0
32     A_banded[0][1] = -1/(dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) # c_0
33
34     # populate middle rows
35     for j in range(1, jmax):
36         # a_j V_{j-1} + b_j V_j + c_j V_{j+1} = d_j
37         A_banded[2][j-1] = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr, 2*(beta-1))
38         - (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) + kappa*j) # a_j
39         A_banded[1][j] = -((1/dt) + 0.5*sigma*sigma*pow(j, 2*beta)*pow(dr, 2*(
40         beta-1)) + j*dr/2 ) # b_j
41         A_banded[0][j+1] = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr, 2*(beta-1))
42         + (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) - kappa*j) # c_j
43     # populate last row
44     # a_jmax V_{jmax-1} + b_jmax V_{jmax} = d_jmax
45     if boundary == 'Neumann':
46         A_banded[2][jmax-1] = -1/dr # a_jmax
47         A_banded[1][jmax] = 1/dr # b_jmax
48     else:
49         A_banded[2][jmax-1] = 0.0 # a_jmax
50         A_banded[1][jmax] = 1.0 # b_jmax
51
52     # Create a vector for multiplication
53     d = np.zeros(jmax+1)
54     d[0] = 1/(dt) * B0ld[0] + C*np.exp(-alpha*(i+0.5)*dt) # d_0

```

```

53     for j in range(1,jmax):
54         aa = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1)) - (1/dr)*
kappa*theta*np.exp(mu*(i+0.5)*dt) + kappa*j) # a_j
55         bb = ((1/dt) - 0.5*sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1)) - j*
dr/2 ) # b_j + 2./dt
56         cc = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1)) + (1/dr)*
kappa*theta*np.exp(mu*(i+0.5)*dt) - kappa*j) # c_j
57         d[j] = - aa*B0ld[j-1] - bb*B0ld[j] - cc*B0ld[j+1] - C*np.exp(-alpha*(i
+0.5)*dt) # d_j
58         d[jmax] = 0.0
59
60     return A_banded, d, l_and_u
61
62 def option_matrix(v0ld, B0ld, i, dr, dt, constants, imax, jmax, rmax, boundary
= 'Dirichlet'):
63
64     sigma, kappa, theta, mu, alpha, beta, C = constants
65
66     A_banded = np.zeros(shape=(3,jmax+1))
67     u, l= 1, 1
68     l_and_u = (1, u)
69
70     # b_0 V_0 + c_0 V_1 = d_0
71     # Call or Put Option
72
73     if Option_Type == "call":
74         A_banded[1][0] = 1 # 1/(dt) + (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt
) # b_0
75         A_banded[0][1] = 0 # -1/(dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) # c_0
76     elif Option_Type == "put":
77         A_banded[1][0] = 1/(dt) + (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) #
b_0
78         A_banded[0][1] = -1/(dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) # c_0
79
80
81     # populate middle rows
82     for j in range(1,jmax):
83         # a_j V_{j-1} + b_j V_j + c_j V_{j+1} = d_j
84         A_banded[2][j-1] = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1))
- (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) + kappa*j) # a_j
85         A_banded[1][j] = -((1/dt) + 0.5*sigma*sigma*pow(j, 2*beta)*pow(dr,2*(
beta-1)) + j*dr/2 ) # b_j
86         A_banded[0][j+1] = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1))
+ (1/dr)*kappa*theta*np.exp(mu*(i+0.5)*dt) - kappa*j) # c_j
87     # populate last row
88     # a_jmax V_{jmax-1} + b_jmax V_{jmax} = d_jmax
89     if boundary == 'Neumann':
90         A_banded[2][jmax-1] = -1/dr # a_jmax
91         A_banded[1][jmax] = 1/dr # b_jmax
92     else:
93         A_banded[2][jmax-1] = 0.0 # a_jmax
94         A_banded[1][jmax] = 1.0 # b_jmax
95
96     # Create a vector for multiplication
97     d = np.zeros(jmax+1)
98

```

```

99     if Option_Type == "call":
100         # Call Option
101         d[0] = B0ld[0]-X # d_0
102     elif Option_Type == "put":
103         #Put Option
104         d[0] = 1/(dt) * v0ld[0] # d_0
105
106     for j in range(1,jmax):
107         aa = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1)) - (1/dr)*
kappa*theta*np.exp(mu*(i+0.5)*dt) + kappa*j) # a_j
108         bb = ((1/dt) - 0.5*sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1)) - j*
dr/2 ) # b_j + 2./dt
109         cc = 0.25*(sigma*sigma*pow(j, 2*beta)*pow(dr,2*(beta-1)) + (1/dr)*
kappa*theta*np.exp(mu*(i+0.5)*dt) - kappa*j) # c_j
110         d[j] = - aa*v0ld[j-1] - bb*v0ld[j] - cc*v0ld[j+1] # d_j
111
112     if Option_Type == "call":
113         # Call Option
114         d[jmax] = 0.0
115     elif Option_Type == "put":
116         # Put Option
117         d[jmax] = X - B0ld[jmax]
118
119
120     return A_banded, d, l_and_u
121
122 def crankNicolson_EuroPut(constants, imax, jmax, rmax, boundary = 'Dirichlet')
:
123
124     sigma, kappa, theta, mu, alpha, beta, C = constants
125
126     ## calculate the step size and then setup storage for the value
127     dr = rmax / jmax
128     dt = T / imax
129
130     r = np.array([j*dr for j in range(jmax+1)])
131     t = np.array([i*dt for i in range(imax+1)])
132     BNew, B0ld = [np.full(jmax+1, F)]*2
133
134     for i in range(imax-1,-1,-1):
135         A_banded, d, l_and_u = Bond_matrix(B0ld, i, dr, dt, constants, imax,
jmax, rmax, boundary)
136         BNew = solve_banded(l_and_u,A_banded,d)
137         B0ld = np.copy(BNew)
138
139     return r,BNew
140
141 def i_j_analysis(N):
142
143     i_j = np.array([100* 2**x for x in range(1,N)])
144
145     B0_old = 0
146     V0_old = 0
147     diff_old = 1
148     for imax in i_j:
149         start_time = time.time()

```

```

150     # r, BNew = crankNicolson_EuroPut(constants, imax, 100, r_max,
boundary = 'Neumann')
151     r,vNew, v_T1, BNew = finiteDiff_AmericanPut(constants, imax, 100,
r_max, boundary = 'Neumann', penalty_method_bool = True)
152     end_time = time.time()
153     elapsed_time = end_time - start_time
154     # BO_new = np.interp(r_0,r,BNew)
155     VO_new = np.interp(r_0, r, vNew)
156     diff = VO_new - VO_old
157     # diff = BO_new - BO_old
158     # print("imax: ", imax, "B(r_0=0,T) :=", BO_new, "Time: ",
elapsed_time, "diff: ", diff, "diff ratio: ", diff_old/diff)
159     print("imax: ", imax, "V(r_0=0,T) :=", VO_new, "Time: ", elapsed_time,
"diff: ", diff, "diff ratio: ", diff_old/diff)
160     # BO_old = BO_new
161     VO_old = VO_new
162     diff_old = diff
163
164     BO_old = 0
165     VO_old = 0
166     diff_old = 1
167     for jmax in i_j:
168         start_time = time.time()
169         # r, BNew = crankNicolson_EuroPut(constants, 100, jmax, r_max,
boundary = 'Neumann')
170         r,vNew, v_T1, BNew = finiteDiff_AmericanPut(constants, 100, jmax,
r_max, boundary = 'Neumann', penalty_method_bool = True)
171         end_time = time.time()
172         elapsed_time = end_time - start_time
173         # BO_new = np.interp(r_0,r,BNew)
174         VO_new = np.interp(r_0, r, vNew)
175         diff = VO_new - VO_old
176         # diff = BO_new - BO_old
177         # print("jmax: ", jmax, "B(r_0=0,T) :=", BO_new, "Time: ",
elapsed_time, "diff: ", diff, "diff ratio: ", diff_old/diff)
178         print("jmax: ", jmax, "V(r_0=0,T) :=", VO_new, "Time: ", elapsed_time,
"diff: ", diff, "diff ratio: ", diff_old/diff)
179         # BO_old = BO_new
180         VO_old = VO_new
181         diff_old = diff
182
183         BO_old = 0
184         VO_old = 0
185         diff_old = 1
186         for max in i_j:
187             start_time = time.time()
188             # r, BNew = crankNicolson_EuroPut(constants, max, max, r_max, boundary
= 'Neumann')
189             r,vNew, v_T1, BNew = finiteDiff_AmericanPut(constants, max, max, r_max
, boundary = 'Neumann', penalty_method_bool = True)
190             end_time = time.time()
191             elapsed_time = end_time - start_time
192             # BO_new = np.interp(r_0,r,BNew)
193             VO_new = np.interp(r_0, r, vNew)
194             diff = VO_new - VO_old
195             # diff = BO_new - BO_old

```

```

196         # print("max: ", max, "B(r_0=0,T) :=", B0_new, "Time: ", elapsed_time,
    "diff: ", diff, "diff ratio: ", diff_old/diff)
197         print("max: ", max, "V(r_0=0,T) :=", V0_new, "Time: ", elapsed_time, "
diff: ", diff, "diff ratio: ", diff_old/diff)
198         # B0_old = B0_new
199         V0_old = V0_new
200         diff_old = diff
201
202 def time_grid_shifter(t, dt, T_1):
203     T1_nearest_index = np.abs(t - T_1).argmin()
204     closest_grid_value = t[T1_nearest_index]
205
206     # If the closest grid value is less than T_1, we need to add the
    difference to the grid
207     # dt_1 is the new first time step and is smaller
208     # dt_2 is the new second time step and is bigger
209     if closest_grid_value < T_1:
210         t[1:-1] = t[1:-1] + np.abs(closest_grid_value - T_1)
211         dt_upper = dt - np.abs(closest_grid_value - T_1)
212         dt_lower = dt + np.abs(closest_grid_value - T_1)
213
214     # If the closest grid value is more than T_1, we need to subtract the
    difference to the grid
215     # dt_1 is the new first time step and is bigger
216     # dt_2 is the new second time step and is smaller
217     elif closest_grid_value > T_1:
218         t[1:-1] = t[1:-1] - np.abs(closest_grid_value - T_1)
219         dt_upper = dt + np.abs(closest_grid_value - T_1)
220         dt_lower = dt - np.abs(closest_grid_value - T_1)
221
222     # update the value of the grid point at T_1 to be equal to exactly T_1
    just in case
223     t[T1_nearest_index] = T_1
224
225     return t, dt_upper, dt_lower, T1_nearest_index
226
227
228 def finiteDiff_AmericanPut(constants, imax, jmax, r_max, boundary = 'Neumann',
    penalty_method_bool = False):
229
230     sigma, kappa, theta, mu, alpha, beta, C = constants
231
232     ## calculate the step size and then setup storage for the value
233     dr = r_max / jmax
234     dt = T / imax
235
236     vNew, vOld = [np.zeros(jmax+1)]*2
237     BNew, BOld = [np.full(jmax+1, F)]*2
238
239     # calculate the values of r_j and t^i and check they work as expected
240     r = np.array([j*dr for j in range(jmax+1)])
241     t = np.array([i*dt for i in range(imax+1)])
242
243     # Find the index of the grid point closest to T_1 and adjust the grid to
    match up with T_1
244     t, dt_upper, dt_lower, T1_nearest_index = time_grid_shifter(t, dt, T_1)

```

```

245
246     #Loop back in time from T to T_1 to calculate the value of the Bond at t=
T_1
247     for i in range(imax-1,T1_nearest_index-1,-1):
248         # Use the adjusted time step for the first step and the normal step
for the remainder
249         dt = T / imax if i != imax-1 else dt_upper
250
251         A_banded, d, l_and_u = Bond_matrix(BOld, i, dr, dt, constants, imax,
jmax, r_max, boundary)
252         BNew = solve_banded(l_and_u,A_banded,d)
253         BOld = np.copy(BNew)
254
255     # Set the boundary condition of the value of the optiona at T_1
256
257     if Option_Type == "call":
258         #Call Option
259         vOld = np.maximum(-X + BNew,0)
260         vNew = np.maximum(-X + BNew,0)
261     elif Option_Type == "put":
262         #Put Option
263         vOld = np.maximum(X - BNew,0)
264         vNew = np.maximum(X - BNew,0)
265
266
267     # Save the value of the option at T_1
268     v_T1 = vOld
269
270     # next loop back through time from t=T_1 to t=0 calculating both the Bond
value and Option value at each step
271     for i in range(T1_nearest_index-1,-1,-1):
272         # use the adjusted time step for the final step and the normal time
step for the rest
273         dt = T / imax if i !=0 else dt_lower
274
275         # Calculate Bond value and next time step
276         AB_banded, d, l_and_u = Bond_matrix(BOld, i, dr, dt, constants, imax,
jmax, r_max, boundary)
277         BNew = solve_banded(l_and_u,AB_banded,d)
278
279         # Calculate option value and next time step #
280         AV_banded, d, l_and_u = option_matrix(vOld, BOld, i, dr, dt, constants
, imax, jmax, r_max, "Dirichlet")
281         vNew = solve_banded(l_and_u,AV_banded,d)
282
283         if penalty_method_bool == True:
284             rho = 1.e8
285             tol = 1e-8
286             maxiter = 50
287             vNew = penalty_method(vNew, rho, tol, maxiter, l_and_u, AV_banded,
d, BNew, i)
288
289         elif Option_Type == "call":
290             vNew = np.maximum(vNew, BNew-X)
291         elif Option_Type == "put":
292             vNew = np.maximum(vNew, X-BNew)

```

```

293
294     # if i % 50 == 0:
295     #     plt.plot(r, vNew)
296     #     plt.plot(r, v_T1)
297     #     plt.xlim(0, 0.1)
298     #     plt.ylim(-0.4, 4)
299     #     plt.show()
300
301     vOld = np.copy(vNew)
302     BOld = np.copy(BNew)
303
304     return r, vNew, v_T1, BNew
305
306 def penalty_method(vNew, rho, tol, maxiter, l_and_u, AV_banded, d, BNew, i):
307     A_banded_dash = np.copy(AV_banded)
308     d_dash = np.copy(d)
309
310     if Option_Type == "call":
311         # Call Option
312         sign = -1
313     elif Option_Type == "put":
314         # Put Option
315         sign = 1
316
317     sign = -1 if Option_Type == "call" else 1
318
319     # Penalty method to ensure that the option value is always greater than
320     # the intrinsic value
321     for q in range(maxiter):
322
323         # check which rows to adjust with a penalty
324         for j in range(1, jmax):
325
326             # check if  $P < X - S$ , if true apply penalty
327             if vNew[j] < (sign*(X - BNew[j])):
328                 #  $b'_j = b_j - \rho$ 
329                 A_banded_dash[1][j] = AV_banded[1][j] - rho
330                 #  $d'_j = d_j - \rho*(X - S)$ 
331                 d_dash[j] = d[j] - rho*(sign*(X - BNew[j]))
332
333             # otherwise revert to original A_banded and d
334             else:
335                 #  $b'_j = b_j$ 
336                 A_banded_dash[1][j] = AV_banded[1][j]
337                 #  $d'_j = d_j$ 
338                 d_dash[j] = d[j]
339
340         # Use banded solver solver to solve problem
341         #  $A' v^{i,q+1} = d'$ 
342         # here  $y \sim v_j^{i,q+1}$ 
343         y = scipy.linalg.solve_banded(l_and_u, A_banded_dash, d_dash)
344
345         # get the change in value from the previous iteration
346         #  $e = v_j^{i,q+1} - v_j^{i,q}$ 
347         e = y - vNew

```

```

348
349         # update the value of vNew
350         vNew = y
351         # check the norm against the tolerance for convergence
352         if np.linalg.norm(e,1) < tol:
353             # print("Solved at step ",i,". Converged after ",q,"iterations
354             .")
355             # exit from the penalty loop
356             break
357         return vNew
358
359 def r_max_analysis(N):
360     r_max_vals = np.array([2**i for i in range(N)])
361
362     for r_max in r_max_vals:
363         j_max = 100*r_max
364         i_max = 100
365         start_time = time.time()
366         r, BNew = crankNicolson_EuroPut(constants, i_max, j_max, r_max,
367         boundary = 'Neumann')
368         end_time = time.time()
369         elapsed_time = end_time - start_time
370         BO_new = np.interp(r_0,r,BNew)
371         print("r_max: ", r_max, "j_max: ", j_max, "Time: ", elapsed_time, "B(
372         r_0=0,T) :=", BO_new)
373
374 if __name__ == "__main__":
375
376
377
378     r, BNew_n = crankNicolson_EuroPut(constants, imax, jmax, r_max, boundary =
379     'Neumann')
380     r, BNew_d = crankNicolson_EuroPut(constants, imax, jmax, r_max, boundary =
381     'Dirichlet')
382     print("B(r_0=0,T) :=", np.interp(r_0, r, BNew_d))
383
384     plt.plot(r,BNew_n, label = 'Neumann Boundary Condition')
385     plt.plot(r,BNew_d, label = 'Dirichlet Boundary Condition')
386     plt.xlabel('r')
387     plt.ylabel('B(r,0,T)')
388     plt.legend()
389     plt.grid()
390     plt.show()
391
392     i_j_analysis(12)
393     r_max_analysis(10)
394
395
396
397     r,vNew, v_T1, BNew = finiteDiff_AmericanPut(constants, imax, jmax, r_max,
398     boundary = 'Neumann', penalty_method_bool = True)

```

```

398     print("V(r_0,0;T_1,T) :=", np.interp(r_0, r, vNew))
399
400     # Filter data for 0.04 < r < 0.1
401     mask = (r > 0.04) & (r < 0.1)
402     r_filtered = r[mask]
403     V_filtered = v_T1[mask]
404
405     plt.plot(r_filtered, V_filtered)
406     plt.show()
407
408
409
410     coefficients = np.polyfit(r_filtered, V_filtered, 1)
411     m, c = coefficients
412
413     # Calculate r-intercept
414     r_intercept = -c / m
415     print("r-intercept:", r_intercept)
416     print("m:", m)
417     print("c:", c)
418
419
420
421     plt.plot(r,vNew, label = 'V(r,0;T_1,T)')
422     plt.plot(r,v_T1, label = 'V(r,T_1;T_1,T)')
423     plt.xlim(0,0.1)
424     plt.ylim(-0.4,8)
425     plt.xlabel('r')
426     plt.ylabel('Option Value V')
427     plt.legend()
428     plt.grid()
429     plt.show()

```